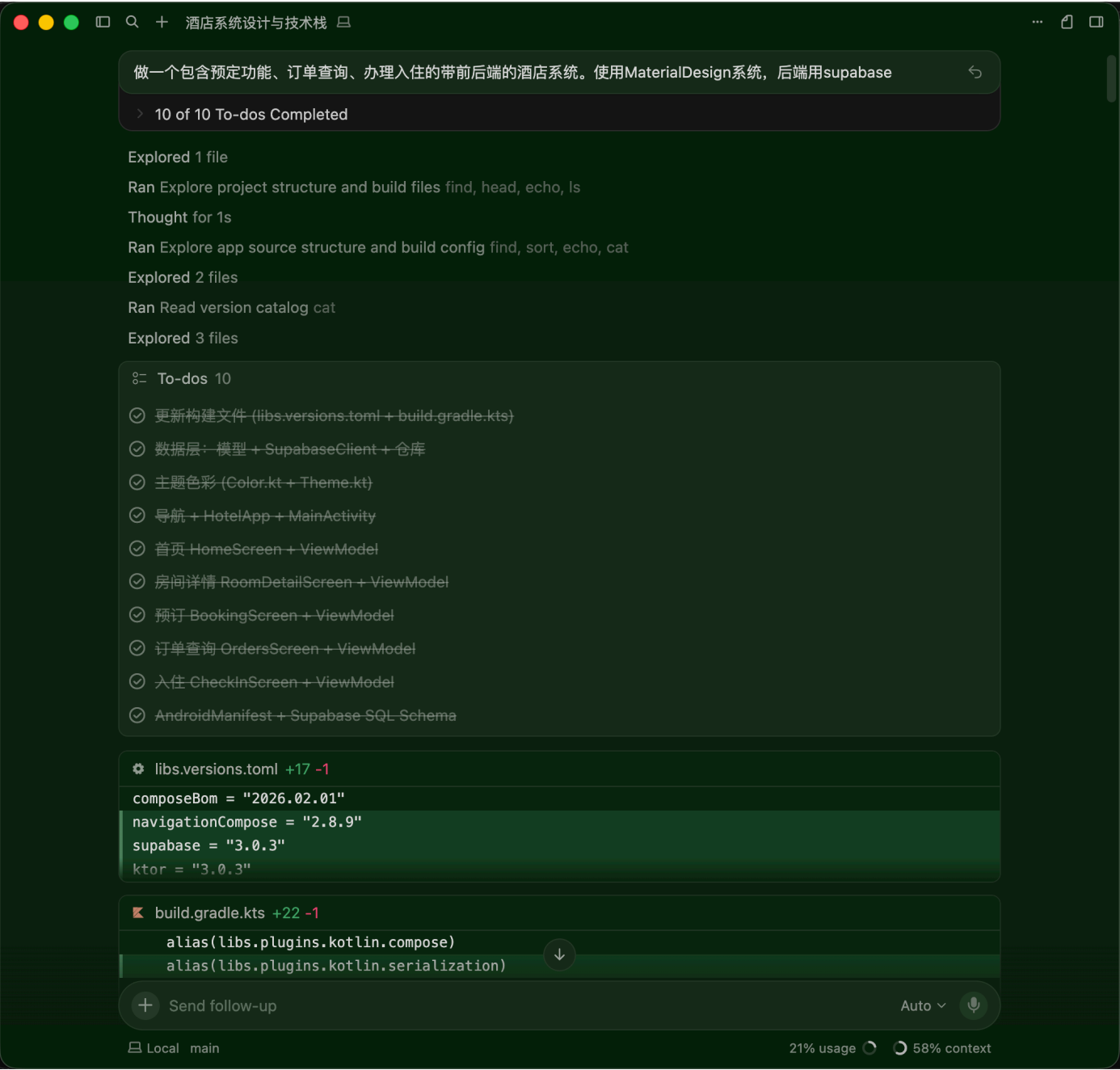


项目记录

1. 完整可运行代码

<https://github.com/DanteAndroid/HotelDemo>

2. 与AI沟通的原始会话记录



酒店系统设计与技术栈

10 of 10 To-dos Completed

使用前配置 Supabase

第一步：建库

登录 [Supabase Dashboard](#)，在 SQL Editor 中执行 `SUPABASE_SCHEMA.sql` 中的所有 SQL — 它会自动创建 `rooms`、`bookings` 两张表并插入 6 条示例房间数据。

第二步：配置 Key

打开 `app/build.gradle.kts`，替换第 18–19 行：

```
buildConfigField("String", "SUPABASE_URL", "\"https://你的项目.supabase.co\"")
buildConfigField("String", "SUPABASE_ANON_KEY", "\"你的 anon key\"")
```

Project URL 和 anon key 在 Supabase 控制台 → Settings → API 中找到。

项目结构总览

22 个 Kotlin 文件，共 3 大功能模块：

- 预订流程 `HomeScreen` → `RoomDetailScreen` → `BookingScreen` → `BookingSuccessScreen`
- 订单查询 `OrdersScreen`（按手机号 / 订单号搜索）
- 办理入住 `CheckInScreen`（输入订单号 → 核对信息 → 确认入住）

技术栈：

- UI：Jetpack Compose + Material Design 3（深青绿 + 金琥珀配色方案，支持 Dynamic Color / 暗色模式）
- 后端：Supabase Postgrest（无需认证，匿名读写）
- 图片：Coil 3 + OkHttp
- 导航：Navigation Compose（底部导航栏 + 页面栈管理）
- 序列化：kotlinx.serialization

执行完了，key也配置好了。可以运行了吗？

正在编译项目以确认能否运行。

Ran Gradle assembleDebug to verify build cd, ./gradlew

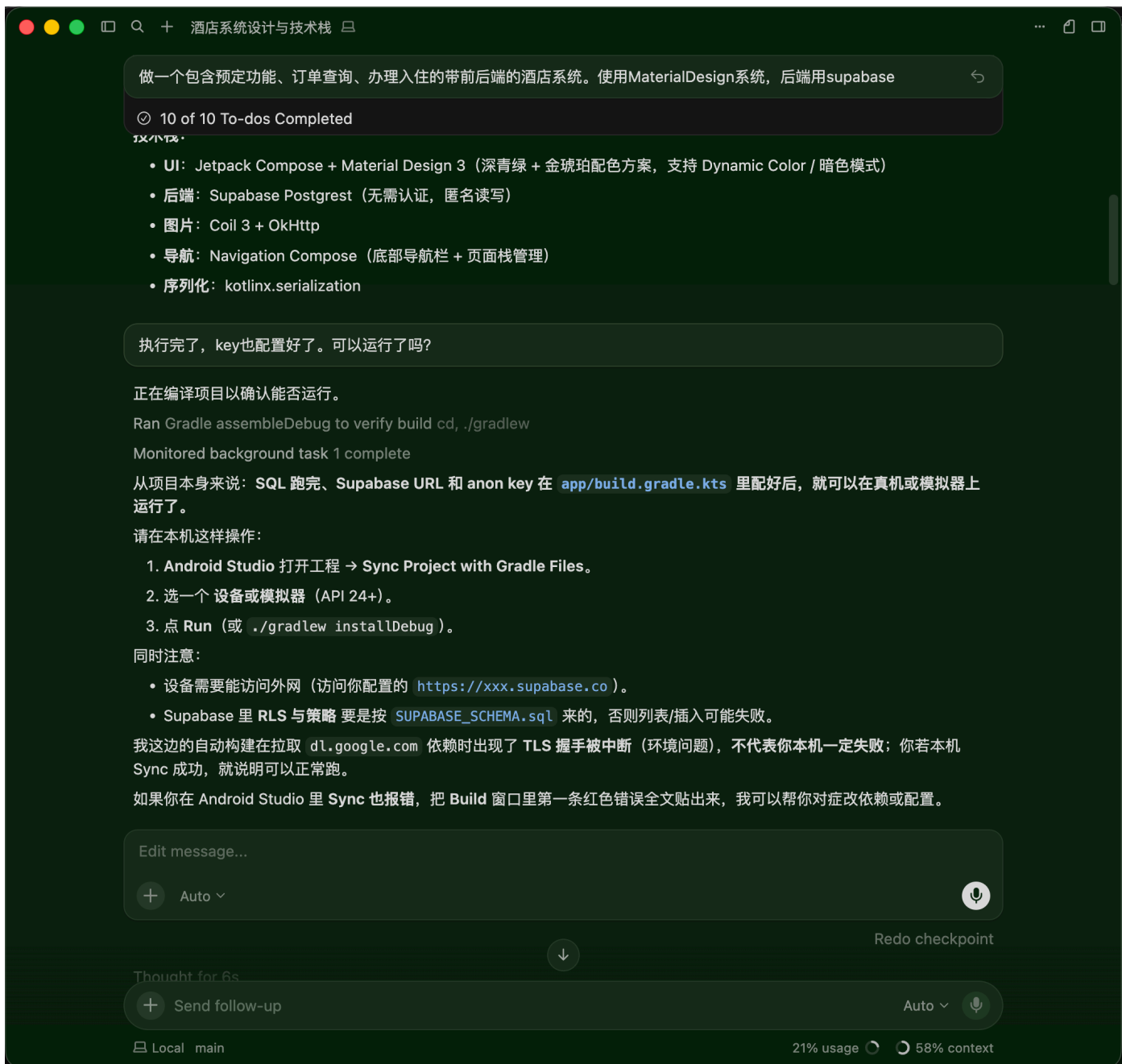
+

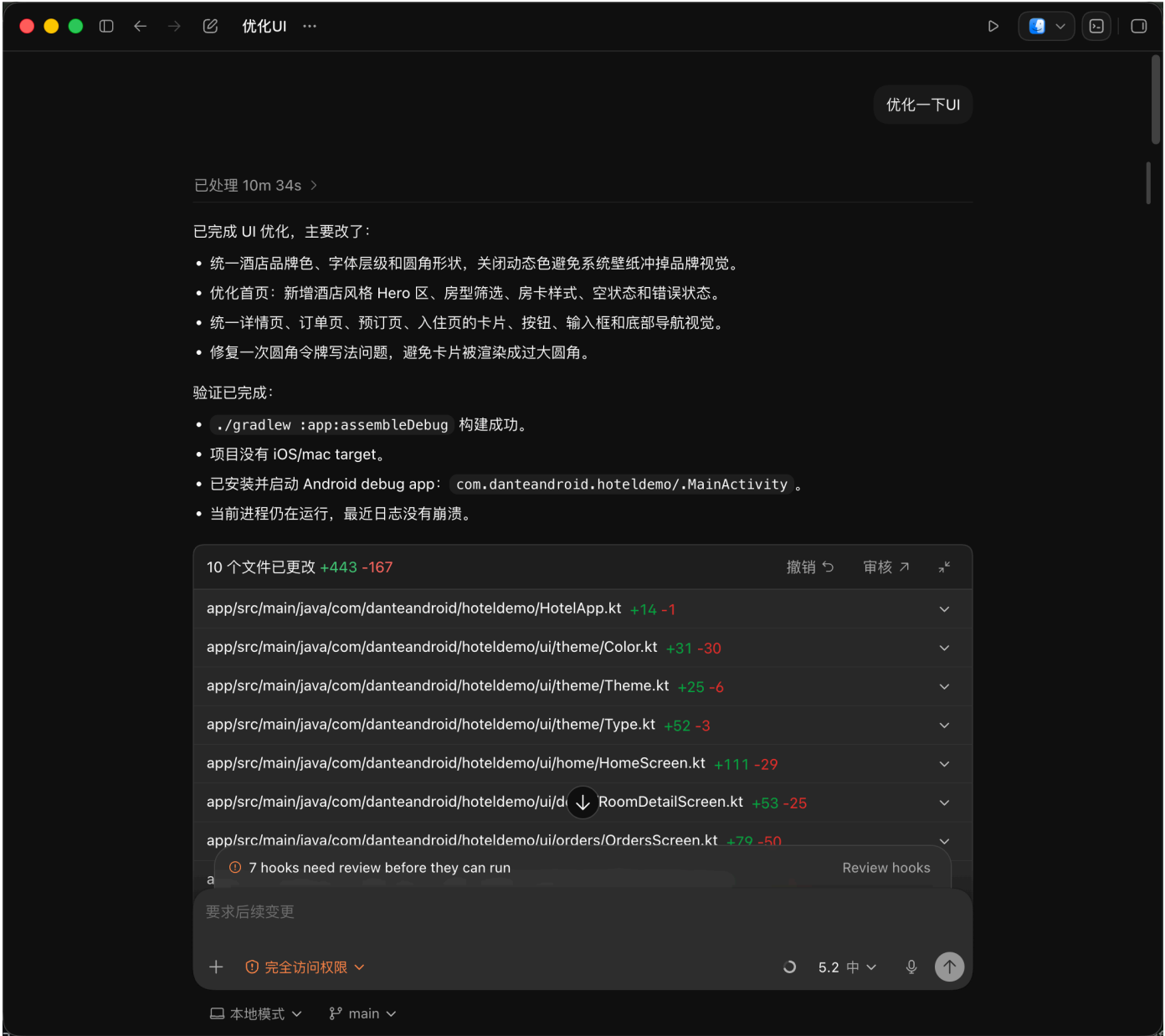
Send follow-up

Auto

🎤

Local main21% usage58% context









- 把底部导航改成覆盖层 (overlay), 不再参与 Scaffold 的布局计算:
`app/src/main/java/com/danteandroid/hoteldemo/HotelApp.kt`
- 底部预留高度从 80.dp -> 0.dp 改为动画收放 (animatedDpAsState), 配合底部栏淡出滑出, 进入详情时不再“闪一下/跳一下”: `app/src/main/java/com/danteandroid/hoteldemo/HotelApp.kt`

1 个文件已更改 撤销 ↶ 审核 ↷ ↕

我先看一下 `RoomDetailViewModel` 和 `RoomRepository` 的实现，确认详情页是不是每次都走 Supabase 网络请求、有没有本地缓存/重复请求，然后给出具体的慢点来源和优化方式。

已处理 14s

现在详情页每次进入都会发一次 Supabase 的 PostgREST 查询：

- app/src/main/java/com/danteandroid/hoteldemo/ui/detail/RoomDetailViewModel.kt 里 init { loadRoom() } 会调用 RoomRepository.getRoomById(roomId)
- app/src/main/java/com/danteandroid/hoteldemo/data/repository/RoomRepository.kt 的 getRoomById() 是 postgreSql["rooms"].select... }, 这是网络请求

查询订单里的订单加个办理入住按钮，同时调整订单的UI（现在太挤了）

收到。我会在订单卡片里加“办理入住”按钮（对可办理的状态显示），点击后跳到“办理入住”页并自动填入订单号；同时把订单卡片重新排版加大留白，减少信息拥挤。改完会重新构建并启动 app 验证。

📄 已编辑 2 个文件,已探索 3 个文件

我接着改两块：1) 订单页把卡片拉开留白，信息分组更清楚；2) 每条订单加“办理入住”按钮，点击后跳到办理入住页并自动带订单号。

📄 已编辑 2 个文件

订单 UI 和“办理入住”按钮都接上了，并且会把订单号带到办理入住页自动查询。现在我跑构建并在模拟器里启动验证跳转链路。

📄 已运行 ./gradlew :app:assembleDebug && /Users//Library/Android/sdk/platform-tools/adb install -r app/b...

我用 adb 自动点到底部“我的订单”，截一张图确认卡片更松、按钮出现；然后点“办理入住”按钮进入办理入住页，再截图确认订单号已自动填入并开始查询。

📄 已运行 /Users//Library/Android/sdk/platform-tools/adb shell input tap 540 2245 && sleep 0.8 && /Users//l...



我发现一个必修点：现在 CheckIn 的路由带了可选参数 `heckin?code={code}`，底部 tab 点击时不能直接拿这个字符串去 navigate，否则会把 `{code}` 当字面量。我先把底部 tab 的跳转改成用

`Screen.CheckIn.createRoute()`

📄 7 hooks need review before they can run

Review hooks

要求后续变更

+

🔒 完全访问权限

🔄 5.2 中

🔍

⬆

📄 本地模式 main



从我的订单里办入住成功后，点击我的订单没反应了。

我先复现并确认当前路由是不是带了 `?code=...` 导致底部栏的“选中/可点击”判断失效；然后把底部栏显示与选中判断改成按 `route` 前缀（`home/orders/checkin`）匹配，避免参数变化把导航搞坏。改完会重新构建并启动验证。

📁 已探索 2 个文件

我定位到一个高风险点：`CheckIn` 现在有参数路由，`currentDestination?.route` 在运行时可能是 `checkin?code=XXXX`，但我们用“完全相等”来判断选中/显示，这会导致底部栏状态异常，进而出现你说的“点了没反应”。我把这些判断改成按 `route` 前缀匹配（`home/orders/checkin`），不受 `query` 参数影响。

✏️ 已编辑 1 个文件

我还要修一个细节：`bottomNavRouteBase` 现在用 `item.screen.route` 当 key（其中 `CheckIn` 是 `checkin?code={code}`），但导航时你实际可能用的是 `checkin`。我把映射改成按 `Screen` 类型直接写常量，保证稳。然后跑构建安装启动复现点击订单。

📁 已编辑 1 个文件,已运行 1 条命令

我用 `adb` 走一次“从订单页进办理入住 -> 成功 -> 再点我的订单”，看是否还能正常响应。由于无法保证你后端有可查订单，我先只验证点击响应链路（不依赖查询结果）。

📁 已运行 /Users//Library/Android/sdk/platform-tools/adb shell input tap 540 2245 && sleep 0.6 && /Users/l/...



🔍

7 hooks need review before they can run

Review hooks

要求后续变更

+

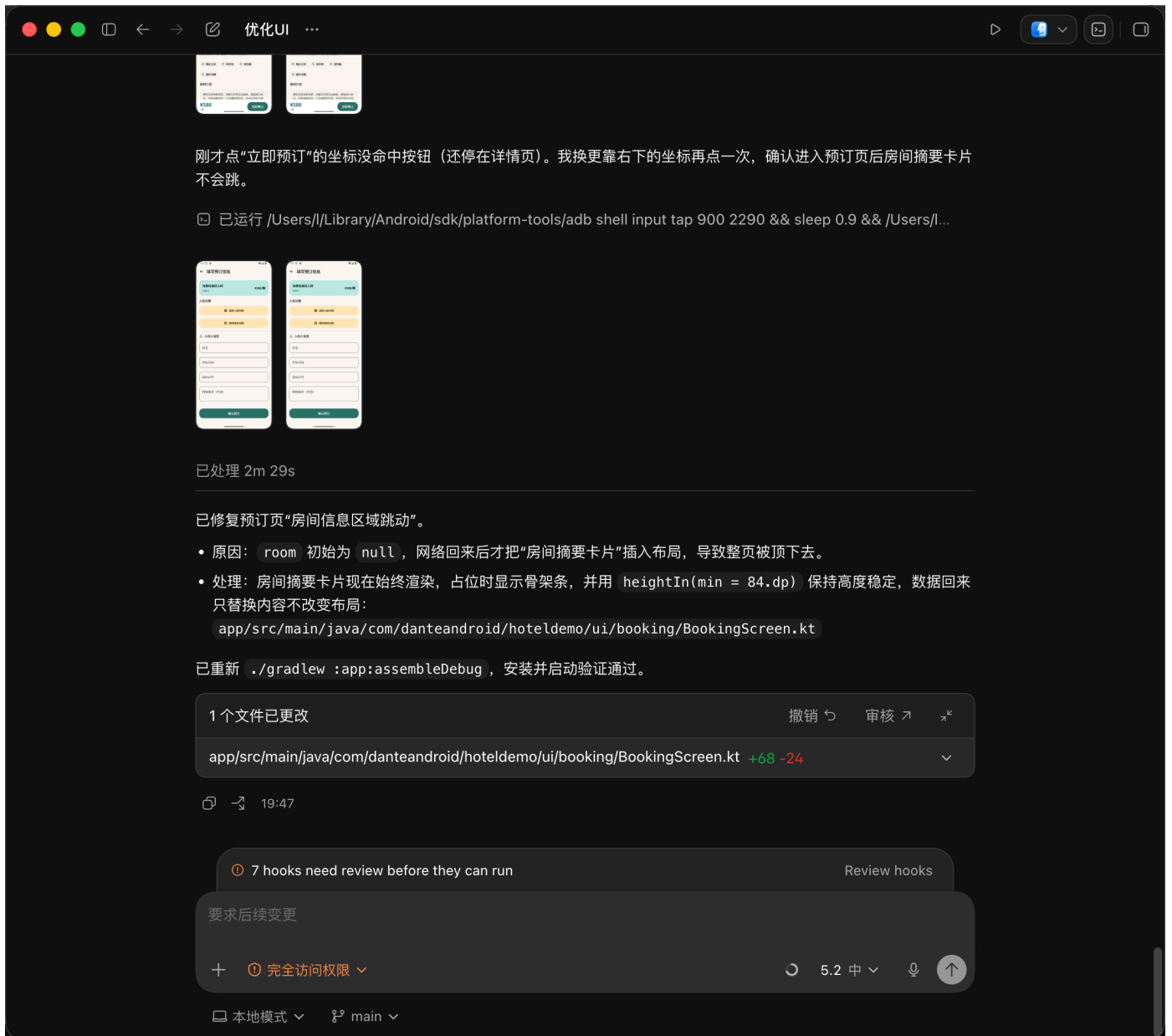
🔒 完全访问权限

5.2 中

🔊

⬆





3. 使用的Skill

无

4. 使用的Agent

Cursor + Codex

5. 过程文档

见聊天记录

6. 产品效果

7. 技术亮点总结

预订状态流转 × Supabase 数据库层能力

酒店系统的核心业务逻辑围绕预订状态流转展开（如：待确认 → 已确认 → 入住中 → 已完成 / 已取消）。

本项目充分利用 **Supabase**（基于 **PostgreSQL**）的数据库原生能力，将关键业务规则下沉到数据库层实现：

能力	说明
RLS（行级安全策略）	按用户角色（客人 / 前台 / 管理员）精细控制数据读写权限，防止越权操作
Functions	封装状态变更逻辑（如确认预订、办理入住），保证业务规则集中、可复用
Triggers	状态变更时自动触发（如更新房间可用状态、写入操作日志），减少客户端逻辑

优势： 业务规则在数据库层强制执行，无论通过 App、管理后台还是直接 API 调用，数据一致性都有保障，同时降低了前端/后端服务的复杂度。